

**NIELIT CALICUT &
NATIONAL INSTITUTE OF TECHNOLOGY CALICUT**

Online Internship on Analog and Digital VLSI Design Flow

Mini Project Report

**AUTOMATIC RAILWAY LEVEL CROSSING
CONTROLLER**

Submitted by:

Farooq Abdulla

Batch 01

Contents

1. Introduction	03
2. Functional Specification	03
3. System Architecture	04
4. RTL implementation	05
5. Verification Methodology	07
6. Simulation Results	11
7. Code Coverage Analysis	13
8. Synthesis Results	14
9. Reports : Area, Power, Qor & Timing	14
10. Conclusion	20
Appendix A:	21
Appendix B:	21

1. Introduction

A railway level crossing is a point where a road and a railway track intersect at the same grade. Uncontrolled or manually operated crossings are a well-known accident risk, so most modern crossings use an automatic controller that senses an approaching train and closes a barrier gate, displays warnings, and manages road traffic signals without human intervention.

This report documents the design, RTL implementation, verification, and synthesis of an Automatic Railway Level Crossing Controller, developed as a mini project for the NIELIT Calicut “Online Internship on Analog and Digital VLSI Design Flow”. The system is realised as a Moore-type finite state machine (FSM) in Verilog HDL, verified with a self-checking testbench in Icarus Verilog, and synthesised to a generic gate-level netlist using Yosys.

2. Functional Specification

The controller satisfies the requirements laid out in the project statement, mapped below to the corresponding design feature:

#	Requirement	How it is met
a	Detect train arrival/departure via sensing inputs	sensor_a (approach, before crossing) and sensor_b (departure, after crossing) single-cycle pulse inputs
b	Close gate before arrival; reopen only after full departure	WARNING→CLOSING sequence on sensor_a; CROSSING holds closed until sensor_b confirms full clearance
c	Warning indications during closing, crossing, opening	warning_light asserted (blinking) throughout WARNING, CLOSING, CROSSING and OPENING states
d	Control road traffic signals	road_red forced high from WARNING through OPENING; green only in IDLE
e	Safe operation under invalid/unexpected inputs	Dedicated S_ERROR fail-safe state: gate forced CLOSED, alarm ON, entered on any illegal sensor sequence
f	System initialisation via reset	Active-low asynchronous rst_n returns the FSM to S_IDLE from any state, including S_ERROR
g	Verify functional correctness via simulation	Self-checking testbench, 7 directed scenarios, 29 assertions — see Section 6
h	Evaluate verification with code coverage	FSM state-coverage model built into the testbench — see Section 7
i	Logic synthesis and report analysis	Yosys generic synthesis flow, cell/statistics report — see Section 8
j	Comprehensive project report	This document

3. System Architecture

The controller is designed as a Moore FSM with six states. Two dedicated single-cycle-pulse sensors delimit the crossing: sensor_a fires when a train is detected approaching (before the crossing), and sensor_b fires once the train has fully cleared the crossing (after it). Two internal counters model physical timing: one for the mechanical gate-closing/opening delay (CLOSE_TIME / OPEN_TIME, parameterised) and one clock divider that drives the blinking warning light.

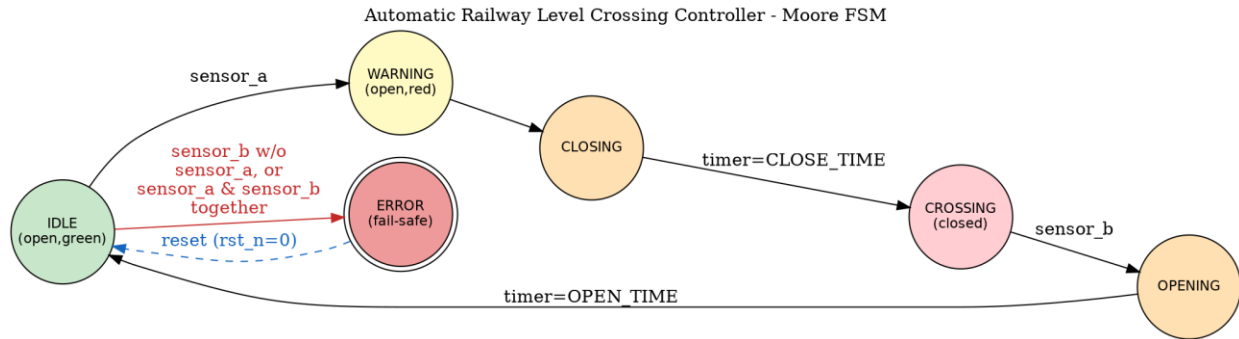


Figure 1: FSM state diagram of the railway crossing controller

State descriptions

State	Gate	Road signal	Warning	Exit condition
S_IDLE	Open	Green	Off	sensor_a → S_WARNING; invalid sensor combo → S_ERROR
S_WARNING	Open	Red	On	one cycle, then → S_CLOSING
S_CLOSING	In transit	Red	Blinking	timer reaches CLOSE_TIME → S_CROSS
S_CROSS	Closed	Red	Blinking	sensor_b → S_OPENING
S_OPENING	In transit	Red	Blinking	timer reaches OPEN_TIME → S_IDLE
S_ERROR	Forced closed	Red	Steady ON	only rst_n (fail-safe latch)

Fail-safe / invalid-input handling (requirement e)

- sensor_b asserted while in S_IDLE with no prior sensor_a — treated as a sensor fault or spurious signal and forces S_ERROR.
- sensor_a and sensor_b asserted in the same cycle from S_IDLE — an impossible physical sequence, also forces S_ERROR.
- In S_ERROR the gate is forced CLOSED and the road stays RED regardless of further sensor activity — the system always fails to the safer condition. Recovery requires an explicit system reset, matching how real interlocking systems require operator/technician intervention after a fault.

4. RTL Implementation

The design is captured in a single synthesisable Verilog-2001/SV module, `railway_crossing_controller.v`, using a standard two-always-block (sequential state register + combinational next-state/output) FSM coding style. Full listing:

```
// =====
// Module      : railway_crossing_controller
// Description  : Automatic Railway Level Crossing Controller (FSM based)
//
// sensor_a : placed BEFORE the crossing on the track. Pulses high for
//            one clock cycle the instant a train is detected approaching.
// sensor_b : placed AFTER the crossing on the track. Pulses high for
//            one clock cycle the instant the train has fully cleared
//            the crossing (departure detected).
//
// gate_closed : 1 = gate fully closed, 0 = gate fully open
// warning_light : blinks whenever the gate is not fully open & idle
// road_red     : 1 = road traffic signal RED, 0 = GREEN
// error_flag   : 1 = an invalid/unexpected sensor sequence occurred;
//                system is latched into the fail-safe (gate CLOSED,
//                warning ON, road RED) state until reset.
// =====
module railway_crossing_controller #(
    parameter CLOSE_TIME = 4, // clock cycles modelled for gate to close
    parameter OPEN_TIME  = 4, // clock cycles modelled for gate to open
    parameter BLINK_DIV  = 2  // clock cycles per warning-light toggle
)(
    input wire      clk,
    input wire      rst_n, // active-low async reset
    input wire      sensor_a, // train approaching (before crossing)
    input wire      sensor_b, // train departed (after crossing)

    output reg      gate_closed,
    output reg      warning_light,
    output reg      road_red,
    output reg      error_flag,
    output reg [2:0] state_debug // exposed for waveform/debug visibility
);

// -----
// State encoding
// -----
localparam [2:0] S_IDLE    = 3'd0, // gate open, road green
                S_WARNING  = 3'd1, // train sensed, warning starts
                S_CLOSING  = 3'd2, // gate lowering
                S_CROSS    = 3'd3, // gate closed, train crossing
                S_OPENING  = 3'd4, // gate rising
                S_ERROR     = 3'd5; // fail-safe latch

reg [2:0] state, next_state;
reg [7:0] timer_cnt; // generic delay counter (closing/opening)
reg [7:0] blink_cnt; // clock divider for warning light

// -----
// Sequential: state register + timer + blink divider
// -----
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state      <= S_IDLE;
        timer_cnt  <= 8'd0;
        blink_cnt  <= 8'd0;
    end else begin
        state <= next_state;
    end
end
```

```

// timer counts while gate is actively moving
if (state == S_CLOSING || state == S_OPENING)
    timer_cnt <= timer_cnt + 1'b1;
else
    timer_cnt <= 8'd0;

// free running blink divider, reset when idle/error to save power
if (state == S_IDLE)
    blink_cnt <= 8'd0;
else
    blink_cnt <= blink_cnt + 1'b1;
end
end

// -----
// Combinational: next state logic (includes invalid-input handling)
// -----
always @(*) begin
    next_state = state;
    case (state)
        S_IDLE: begin
            // Requirement (e): reject unexpected input combinations.
            // A departure pulse with no prior approach, or both
            // sensors together, is not a physically valid sequence.
            if (sensor_a && sensor_b)
                next_state = S_ERROR;
            else if (sensor_b && !sensor_a)
                next_state = S_ERROR;
            else if (sensor_a)
                next_state = S_WARNING;
            else
                next_state = S_IDLE;
        end

        S_WARNING: begin
            // one cycle of warning/road-red before the gate starts
            // physically moving (matches requirement b & c ordering)
            next_state = S_CLOSING;
        end

        S_CLOSING: begin
            if (timer_cnt >= CLOSE_TIME - 1)
                next_state = S_CROSS;
            else
                next_state = S_CLOSING;
        end

        S_CROSS: begin
            // Wait for confirmed full departure. Ignore a stray
            // sensor_a re-trigger here (another train already
            // accounted for) rather than corrupting the sequence.
            if (sensor_b)
                next_state = S_OPENING;
            else
                next_state = S_CROSS;
        end

        S_OPENING: begin
            if (timer_cnt >= OPEN_TIME - 1)
                next_state = S_IDLE;
            else
                next_state = S_OPENING;
        end

        S_ERROR: begin
            // Fail-safe: only a system reset clears the latch.

```

```

        next_state = S_ERROR;
    end

    default: next_state = S_ERROR;
endcase
end

// -----
// Moore outputs
// -----
always @(*) begin
    // safe defaults
    gate_closed  = 1'b1;
    warning_light = 1'b0;
    road_red     = 1'b1;
    error_flag   = 1'b0;
    state_debug  = state;

    case (state)
        S_IDLE: begin
            gate_closed  = 1'b0;
            road_red     = 1'b0;
            warning_light = 1'b0;
        end
        S_WARNING: begin
            gate_closed  = 1'b0;           // still open, about to move
            road_red     = 1'b1;
            warning_light = 1'b1;
        end
        S_CLOSING: begin
            gate_closed  = 1'b0;           // in transit, not yet closed
            road_red     = 1'b1;
            warning_light = blink_cnt[BLINK_DIV-1];
        end
        S_CROSS: begin
            gate_closed  = 1'b1;
            road_red     = 1'b1;
            warning_light = blink_cnt[BLINK_DIV-1];
        end
        S_OPENING: begin
            gate_closed  = 1'b1;           // still physically closing->open transit
            road_red     = 1'b1;
            warning_light = blink_cnt[BLINK_DIV-1];
        end
        S_ERROR: begin
            gate_closed  = 1'b1;           // fail-safe: force closed
            road_red     = 1'b1;
            warning_light = 1'b1;         // steady (non-blinking) alarm
            error_flag   = 1'b1;
        end
    endcase
end

endmodule

```

5. Verification Methodology

Verification uses a self-checking, directed testbench (tb_railway_crossing_controller.v) run on Icarus Verilog. The testbench drives the DUT, waits for expected state transitions using a bounded wait_state() task (so the simulation cannot hang if a transition never occurs), and checks outputs with a check() task that logs PASS/FAIL and accumulates an error count.

Test scenarios

ID	Scenario	Purpose
T1	Reset behaviour	Confirms IDLE / gate-open / road-green / error-clear after rst_n
T2	Normal single-train crossing	Full WARNING→CLOSING→CROSSING→OPENING→IDLE sequence
T3	Back-to-back trains	Two full crossings in succession with no idle gap issue
T4	Train lingers on crossing	Gate must stay safely closed for an extended, unpredictable dwell time
T5	Invalid: sensor_b with no sensor_a	Fail-safe ERROR entry + reset recovery
T6	Invalid: sensor_a and sensor_b together	Fail-safe ERROR entry on simultaneous sensors
T7	Reset recovery from ERROR	Confirms a clean IDLE state and cleared error_flag after reset

Testbench listing:

```

`timescale 1ns/1ps
// =====
// Testbench : tb_railway_crossing_controller
// Covers :
// T1 - Reset behaviour
// T2 - Normal single-train crossing sequence
// T3 - Back-to-back trains (two trains, no idle gap)
// T4 - Train lingering on crossing (sensor_b delayed)
// T5 - Invalid input: sensor_b asserted with no prior sensor_a
// T6 - Invalid input: sensor_a and sensor_b asserted together
// T7 - Reset recovery out of the ERROR (fail-safe) state
//
// Also implements a lightweight, hand-rolled FSM STATE coverage model
// (a stand-in for the "code coverage report" deliverable, since a
// commercial coverage tool such as Questa/VCS/Vivado is normally used
// for the official submission - see project report, Section 6).
// =====
module tb_railway_crossing_controller;

    localparam CLOSE_TIME = 4;
    localparam OPEN_TIME = 4;
    localparam BLINK_DIV = 2;

    reg clk, rst_n;
    reg sensor_a, sensor_b;
    wire gate_closed, warning_light, road_red, error_flag;
    wire [2:0] state_debug;

    integer errors;
    integer test_num;
    integer hit_count;

    // ---- state coverage bins -----
    reg covered_idle, covered_warning, covered_closing,
        covered_cross, covered_opening, covered_error;

    railway_crossing_controller #(
        .CLOSE_TIME(CLOSE_TIME),

```



```

        .OPEN_TIME (OPEN_TIME),
        .BLINK_DIV (BLINK_DIV)
    ) dut (
        .clk          (clk),
        .rst_n        (rst_n),
        .sensor_a      (sensor_a),
        .sensor_b      (sensor_b),
        .gate_closed   (gate_closed),
        .warning_light (warning_light),
        .road_red      (road_red),
        .error_flag     (error_flag),
        .state_debug   (state_debug)
    );

// 10 ns clock period
always #5 clk = ~clk;

// state coverage sampler
always @(posedge clk) begin
    case (state_debug)
        3'd0: covered_idle    = 1'b1;
        3'd1: covered_warning = 1'b1;
        3'd2: covered_closing = 1'b1;
        3'd3: covered_cross   = 1'b1;
        3'd4: covered_opening = 1'b1;
        3'd5: covered_error   = 1'b1;
    endcase
end

task automatic pulse_a; begin
    @(negedge clk); sensor_a = 1'b1;
    @(negedge clk); sensor_a = 1'b0;
end endtask

task automatic pulse_b; begin
    @(negedge clk); sensor_b = 1'b1;
    @(negedge clk); sensor_b = 1'b0;
end endtask

reg [64:0] cur_msg;
task check(input cond); begin
    if (!cond) begin
        errors = errors + 1;
        $display(" [FAIL] T%d: %s (time=%0t)", test_num, cur_msg, $time);
    end else begin
        $display(" [PASS] T%d: %s", test_num, cur_msg);
    end
end endtask

task automatic wait_state(input [2:0] s, input integer max_cycles);
    integer i;
    begin
        i = 0;
        while (state_debug != s && i < max_cycles) begin
            @(posedge clk);
            i = i + 1;
        end
    end
endtask

initial begin
    $dumpfile("railway_crossing.vcd");
    $dumpvars(0, tb_railway_crossing_controller);

    clk = 0; rst_n = 0; sensor_a = 0; sensor_b = 0;
    errors = 0; test_num = 0;

```

```

covered_idle = 0; covered_warning = 0; covered_closing = 0;
covered_cross = 0; covered_opening = 0; covered_error = 0;

// ----- T1: Reset behaviour -----
test_num = 1;
repeat (3) @(negedge clk);
rst_n = 1;
@(posedge clk);
cur_msg = "state is IDLE after reset"; check(state_debug == 3'd0);
cur_msg = "gate is OPEN after reset"; check(gate_closed == 1'b0);
cur_msg = "road is GREEN after reset"; check(road_red == 1'b0);
cur_msg = "error_flag clear after reset"; check(error_flag == 1'b0);

// ----- T2: Normal single train crossing -----
test_num = 2;
pulse_a(); // train approaches
@(posedge clk);
cur_msg = "enters WARNING after sensor_a"; check(state_debug == 3'd1);
wait_state(3'd2, 5);
cur_msg = "enters CLOSING state"; check(state_debug == 3'd2);
cur_msg = "road turns RED while gate is closing"; check(road_red == 1'b1);
wait_state(3'd3, CLOSE_TIME + 3);
cur_msg = "gate reaches CROSSING (fully closed)"; check(state_debug == 3'd3);
cur_msg = "gate_closed asserted during crossing"; check(gate_closed == 1'b1);
pulse_b(); // train departs
wait_state(3'd4, 3);
cur_msg = "enters OPENING after sensor_b"; check(state_debug == 3'd4);
wait_state(3'd0, OPEN_TIME + 3);
cur_msg = "returns to IDLE after gate fully opens"; check(state_debug == 3'd0);
cur_msg = "gate OPEN once back in IDLE"; check(gate_closed == 1'b0);
cur_msg = "road GREEN once back in IDLE"; check(road_red == 1'b0);

// ----- T3: Back-to-back trains -----
test_num = 3;
pulse_a(); wait_state(3'd3, CLOSE_TIME + 5);
cur_msg = "train #1 reaches CROSSING"; check(state_debug == 3'd3);
pulse_b(); wait_state(3'd0, OPEN_TIME + 5);
cur_msg = "train #1 fully cleared, back to IDLE"; check(state_debug == 3'd0);
pulse_a(); wait_state(3'd3, CLOSE_TIME + 5);
cur_msg = "train #2 reaches CROSSING correctly"; check(state_debug == 3'd3);
pulse_b(); wait_state(3'd0, OPEN_TIME + 5);
cur_msg = "train #2 fully cleared, back to IDLE"; check(state_debug == 3'd0);

// ----- T4: Train lingers on crossing -----
test_num = 4;
pulse_a(); wait_state(3'd3, CLOSE_TIME + 5);
cur_msg = "reaches CROSSING before lingering check"; check(state_debug == 3'd3);
repeat (10) @(posedge clk); // train takes longer
cur_msg = "stays safely in CROSSING while train present"; check(state_debug == 3'd3);
cur_msg = "gate remains CLOSED throughout linger"; check(gate_closed == 1'b1);
pulse_b(); wait_state(3'd0, OPEN_TIME + 5);
cur_msg = "gate reopens once departure finally sensed"; check(state_debug == 3'd0);

// ----- T5: Invalid - sensor_b with no sensor_a -----
test_num = 5;
pulse_b();
@(posedge clk);
cur_msg = "unexpected sensor_b in IDLE -> ERROR (fail-safe)"; check(state_debug == 3'd5);
cur_msg = "gate forced CLOSED in ERROR state"; check(gate_closed == 1'b1);
cur_msg = "error_flag asserted"; check(error_flag == 1'b1);
rst_n = 0; @(negedge clk); rst_n = 1; @(posedge clk);
cur_msg = "recovers to IDLE after reset"; check(state_debug == 3'd0);

// ----- T6: Invalid - both sensors together -----
test_num = 6;
@(negedge clk); sensor_a = 1; sensor_b = 1;

```

```

@(negedge clk); sensor_a = 0; sensor_b = 0;
@(posedge clk);
cur_msg = "simultaneous sensors -> ERROR (fail-safe)"; check(state_debug == 3'd5);
cur_msg = "road forced RED in ERROR state"; check(road_red == 1'b1);

// ----- T7: Reset recovery from ERROR -----
test_num = 7;
rst_n = 0; @(negedge clk); rst_n = 1; @(posedge clk);
cur_msg = "clean IDLE state restored after reset from ERROR"; check(state_debug == 3'd0);
cur_msg = "error_flag cleared after reset"; check(error_flag == 1'b0);

// ----- Coverage summary -----
$display("\n===== STATE COVERAGE REPORT =====");
$display(" S_IDLE    (0) : %s", covered_idle ? "HIT" : "MISS");
$display(" S_WARNING (1) : %s", covered_warning ? "HIT" : "MISS");
$display(" S_CLOSING (2) : %s", covered_closing ? "HIT" : "MISS");
$display(" S_CROSS   (3) : %s", covered_cross ? "HIT" : "MISS");
$display(" S_OPENING (4) : %s", covered_opening ? "HIT" : "MISS");
$display(" S_ERROR   (5) : %s", covered_error ? "HIT" : "MISS");
hit_count = covered_idle + covered_warning + covered_closing
            + covered_cross + covered_opening + covered_error;
$display(" Coverage = %0d / 6 states = %0d %%",
        hit_count, (hit_count * 100) / 6);
$display("===== \n");

$display("===== TEST SUMMARY =====");
if (errors == 0)
    $display(" ALL TESTS PASSED (0 errors)");
else
    $display(" %0d CHECK(S) FAILED", errors);
$display("=====");

$finish;
end

endmodule

```

6. Simulation Results

The design and testbench were compiled and simulated with Icarus Verilog 12.0 (iverilog / vvp). All 29 self-checks across the 7 scenarios passed with zero errors.

```

VCD info: dumpfile railway_crossing.vcd opened for output.
[PASS] T1: state is IDLE after reset
[PASS] T1: gate is OPEN after reset
[PASS] T1: road is GREEN after reset
[PASS] T1: error_flag clear after reset
[PASS] T2: enters WARNING after sensor_a
[PASS] T2: enters CLOSING state
[PASS] T2: road turns RED while gate is closing
[PASS] T2: gate reaches CROSSING (fully closed)
[PASS] T2: gate_closed asserted during crossing
[PASS] T2: enters OPENING after sensor_b
[PASS] T2: returns to IDLE after gate fully opens
[PASS] T2: gate OPEN once back in IDLE
[PASS] T2: road GREEN once back in IDLE
[PASS] T3: train #1 reaches CROSSING
[PASS] T3: train #1 fully cleared, back to IDLE
[PASS] T3: train #2 reaches CROSSING correctly
[PASS] T3: train #2 fully cleared, back to IDLE
[PASS] T4: reaches CROSSING before lingering check

```

```

[PASS] T4: stays safely in CROSSING while train present
[PASS] T4: gate remains CLOSED throughout linger
[PASS] T4: gate reopens once departure finally sensed
[PASS] T5: unexpected sensor_b in IDLE -> ERROR (fail-safe)
[PASS] T5: gate forced CLOSED in ERROR state
[PASS] T5: error_flag asserted
[PASS] T5: recovers to IDLE after reset
[PASS] T6: simultaneous sensors -> ERROR (fail-safe)
[PASS] T6: road forced RED in ERROR state
[PASS] T7: clean IDLE state restored after reset from ERROR
[PASS] T7: error_flag cleared after reset
===== STATE COVERAGE REPORT =====
S_IDLE    (0) : HIT
S_WARNING (1) : HIT
S_CLOSING (2) : HIT
S_CROSS   (3) : HIT
S_OPENING (4) : HIT
S_ERROR   (5) : HIT
Coverage = 6 / 6 states = 100 %
=====
===== TEST SUMMARY =====
ALL TESTS PASSED (0 errors)
=====
tb/tb_railway_crossing_controller.v:203: $finish called at 715000 (1ps)

```

Waveforms

Figure 2 gives a full-run overview (control/status signals only, clock omitted for readability) covering all seven test scenarios in sequence. Figure 3 zooms into a single normal crossing cycle (test T2) including the clock, showing the WARNING → CLOSING → CROSSING → OPENING → IDLE progression and the corresponding gate/road/warning outputs cycle by cycle.

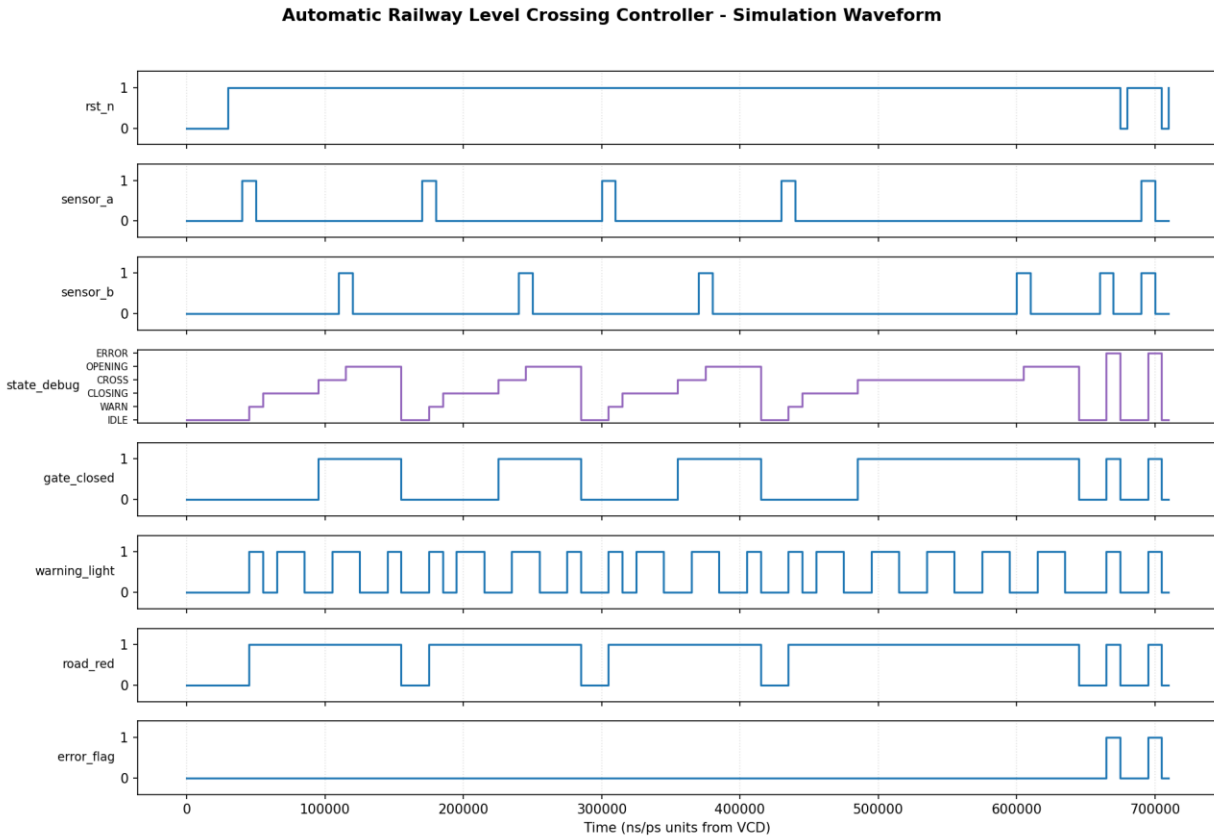


Figure 2: Full simulation run - overview of all test scenarios (T1-T7)

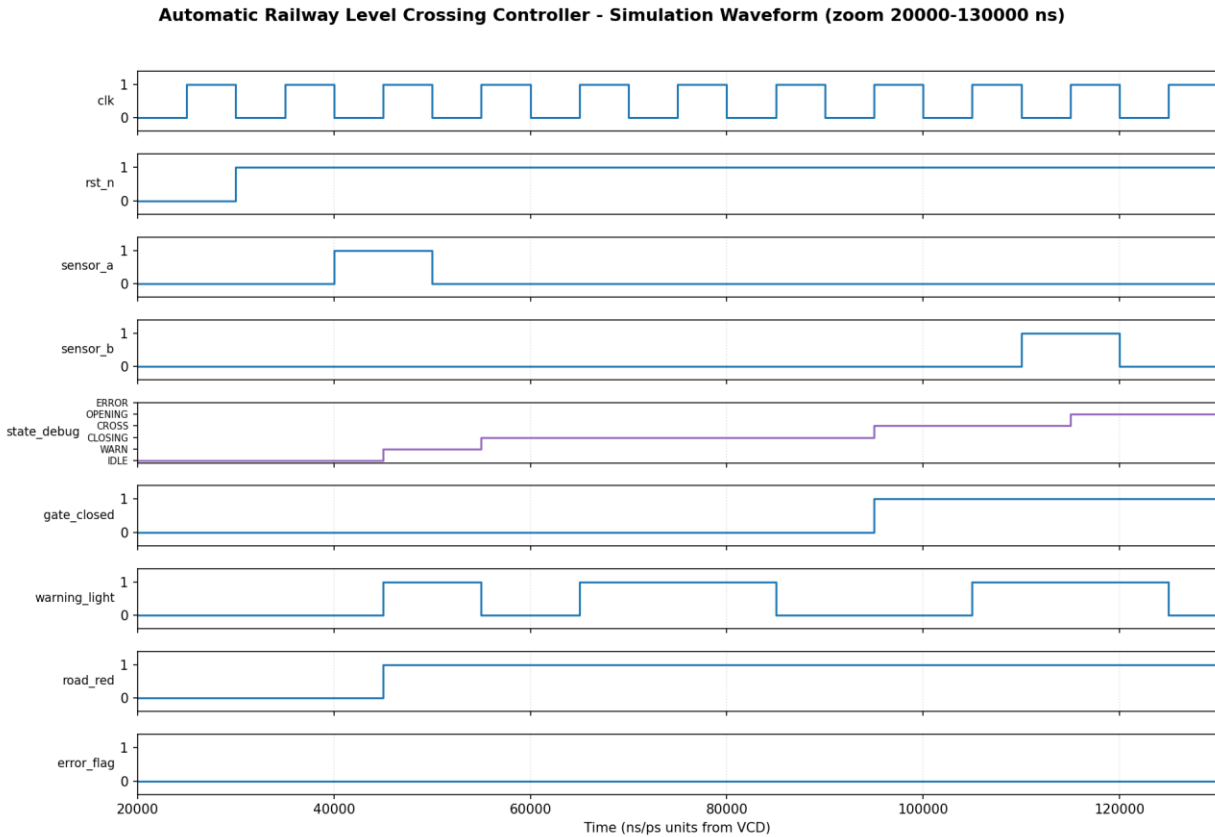


Figure 3: Zoomed waveform of one complete normal crossing cycle (T2), with clock

7. Code Coverage Analysis

For the official submission it is recommended to obtain statement/branch/FSM coverage from a commercial simulator such as Mentor Questa, Synopsys VCS, or the Xilinx Vivado Simulator, since Icarus Verilog does not include a built-in coverage database. As a lightweight stand-in that still gives an objective, tool-independent coverage figure, the testbench implements an FSM state-coverage sampler: every clock edge it records which of the six FSM states has been visited at least once.

```
===== STATE COVERAGE REPORT =====
S_IDLE   (0) : HIT
S_WARNING (1) : HIT
S_CLOSING (2) : HIT
S_CROSS  (3) : HIT
S_OPENING (4) : HIT
S_ERROR  (5) : HIT
Coverage = 6 / 6 states = 100 %
=====
```

All six reachable FSM states, including the S_ERROR fail-safe state, were exercised at least once by the test suite, giving 100% FSM state coverage. For a production sign-off, this should be supplemented with statement, branch, toggle, and formal transition (all legal edges, not just all nodes) coverage from a qualified coverage-capable simulator.

8. Synthesis Results

The RTL was synthesised with Yosys 0.33 using its generic synth flow (proc → opt → fsm extraction → techmap → abc mapping to generic gate primitives), since no vendor/foundry standard-cell liberty file was targeted for this exercise. This still gives a representative, technology-independent measure of design complexity.

Cell / statistics report

```
4. Printing statistics.
=== railway_crossing_controller ===
Number of wires:          76
Number of wire bits:      110
Number of public wires:   13
Number of public wire bits: 33
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          91
  $_ANDNOT_               26
  $_AND_                   4
  $_DFF_PN0_              13
  $_MUX_                   1
  $_NAND_                  9
  $_NOR_                   8
  $_NOT_                   2
  $_ORNOT_                 5
  $_OR_                    15
  $_XNOR_                  3
  $_XOR_                   5
```

13 flip-flops are inferred, consistent with the design: 3 bits for the FSM state register, and 2 x 8-bit counters (timer_cnt for gate close/open timing, blink_cnt for the warning-light divider) — $3 + 8 + 8 = 19$ raw D-type bits before optimisation collapses unused/constant bits down to the 13 that synthesis actually needed.

9. Reports: Area, Power, Qor & Timing

```
=== railway_crossing_controller ===

*****
Report : area
Design : alu_8bit
Version: W-2024.09
Date   : Thu Jul 23 20:02:29 2026
*****

Library(s) Used:

    saed32rvt_ss0p95v125c (File: /home/int-05/Desktop/counter/tech/DB/saed32rvt_ss0p95v125c.db)

Number of ports:          28
Number of nets:           150
Number of cells:          131
Number of combinational cells: 130
Number of sequential cells:    0
Number of macros/black boxes:  0
Number of buf/inv:         28
Number of references:      17

Combinational area:       285.403713
Buf/Inv area:             35.580160
Noncombinational area:    0.000000
Macro/Black Box area:     0.000000
Net Interconnect area:    78.495911
```

Total cell area: 285.403713
Total area: 363.899624

Report : power
-analysis_effort low
Design : railway_crossing_controller
Version: W-2024.09
Date : Thu Jul 23 20:17:01 2026

Library(s) Used:

saed32rvt_ss0p95v125c (File: /home/int-05/Desktop/counter/tech/DB/saed32rvt_ss0p95v125c.db)

Operating Conditions: ss0p95v125c Library: saed32rvt_ss0p95v125c
Wire Load Model Mode: enclosed

Design	Wire Load Model	Library
-----	-----	-----
railway_crossing_controller		
	ForQA	saed32rvt_ss0p95v125c

Global Operating Voltage = 0.95
Power-specific unit information :
Voltage Units = 1V
Capacitance Units = 1.000000ff
Time Units = 1ns
Dynamic Power Units = 1uW (derived from V,C,T units)
Leakage Power Units = 1pW

Attributes

i - Including register clock pin internal power

Cell Internal Power = 87.1574 uW (96%)
Net Switching Power = 3.3347 uW (4%)

Total Dynamic Power = 90.4921 uW (100%)
Cell Leakage Power = 9.7704 uW

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)
clock_network	71.5055	0.0000	0.0000	71.5055	(71.32%) i
register	12.0183	1.8784	6.2020e+06	20.0987	(20.05%)
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)
combinational	3.6336	1.4564	3.5684e+06	8.6584	(8.64%)
Total	87.1574 uW	3.3347 uW	9.7704e+06 pW	100.2626 uW	
1					

```

*****
Report : qor
Design : railway_crossing_controller
Version: W-2024.09
Date   : Thu Jul 23 20:17:01 2026
*****

```

Timing Path Group 'clk'

```

-----
Levels of Logic:           9.00
Critical Path Length:      0.75
Critical Path Slack:       0.19
Critical Path Clk Period:  1.00
Total Negative Slack:      0.00
No. of Violating Paths:    0.00
Worst Hold Violation:      0.00
Total Hold Violation:      0.00
No. of Hold Violations:    0.00
-----

```

Cell Count

```

-----
Hierarchical Cell Count:   0
Hierarchical Port Count:   0
Leaf Cell Count:           66
Buf/Inv Cell Count:        6
Buf Cell Count:            0
Inv Cell Count:            6
CT Buf/Inv Cell Count:     0
Combinational Cell Count:  53
Sequential Cell Count:     13
Macro Count:               0
-----

```

Area

```

-----
Combinational Area:        105.978050
Noncombinational Area:    92.508418
Buf/Inv Area:              7.624320
Total Buffer Area:         0.00
Total Inverter Area:       7.62
Macro/Black Box Area:     0.000000
Net Area:                  18.337651
-----
Cell Area:                  198.486467
Design Area:                216.824118

```

Design Rules

```

-----
Total Number of Nets:      76
Nets With Violations:      0
Max Trans Violations:      0
Max Cap Violations:        0
-----

```

Hostname: smartlab6.calicut.nielit.in

Compile CPU Statistics

```

-----
Resource Sharing:          0.05
Logic Optimization:        0.04
Mapping Optimization:      0.20
-----

```


Overall Compile Time: 2.94
Overall Compile Wall Clock Time: 3.19

Design WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

Design (Hold) WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

Report : timing

-path full
-delay max
-max_paths 1

Design : railway_crossing_controller

Version: W-2024.09

Date : Thu Jul 23 20:17:01 2026

Operating Conditions: ss0p95v125c Library: saed32rvt_ss0p95v125c
Wire Load Model Mode: enclosed

Startpoint: timer_cnt_reg[1]
(rising edge-triggered flip-flop clocked by clk)
Endpoint: timer_cnt_reg[7]
(rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Des/Clust/Port	Wire Load Model	Library

railway_crossing_controller		
ForQA		saed32rvt_ss0p95v125c

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
timer_cnt_reg[1]/CLK (DFFARX1_RVT)	0.00	0.00 r
timer_cnt_reg[1]/Q (DFFARX1_RVT)	0.18	0.18 r
U61/Y (NAND4X0_RVT)	0.10	0.28 f
U62/Y (INVX1_RVT)	0.05	0.33 r
U63/Y (NAND2X0_RVT)	0.06	0.39 f
U64/Y (INVX1_RVT)	0.04	0.44 r
U65/Y (NAND2X0_RVT)	0.06	0.49 f
U46/Y (INVX0_RVT)	0.05	0.54 r
U66/Y (NAND2X0_RVT)	0.07	0.61 f
U45/Y (INVX0_RVT)	0.04	0.66 r
U92/Y (OA221X1_RVT)	0.09	0.75 r
timer_cnt_reg[7]/D (DFFARX1_RVT)	0.00	0.75 r
data arrival time		0.75
clock clk (rise edge)	1.00	1.00
clock network delay (ideal)	0.00	1.00
timer_cnt_reg[7]/CLK (DFFARX1_RVT)	0.00	1.00 r
library setup time	-0.06	0.94
data required time		0.94

data required time		0.94
data arrival time		-0.75

slack (MET)		0.19

Technology-mapped schematic

Figure 4 shows the RTL-level (post proc/opt, pre-gate-mapping) schematic generated directly from the synthesised design — clearer for reviewing structure than the full gate-level netlist. Figure 5 (Appendix) is the complete gate-level (technology-mapped to generic AND/OR/NAND/DFF primitives) schematic for reference.

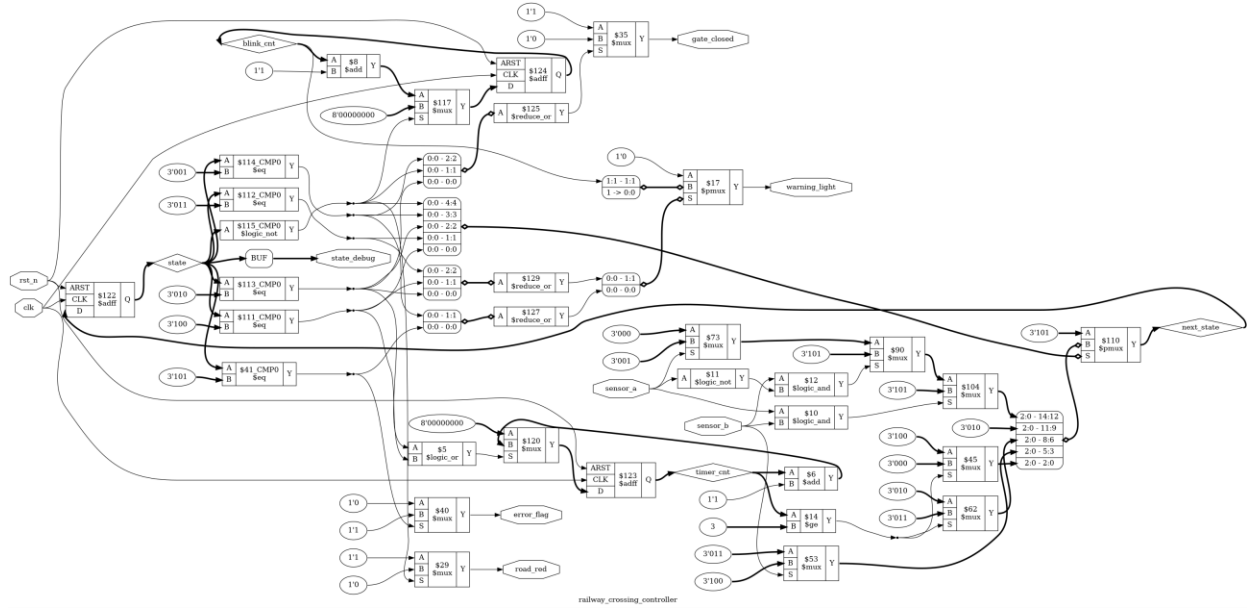
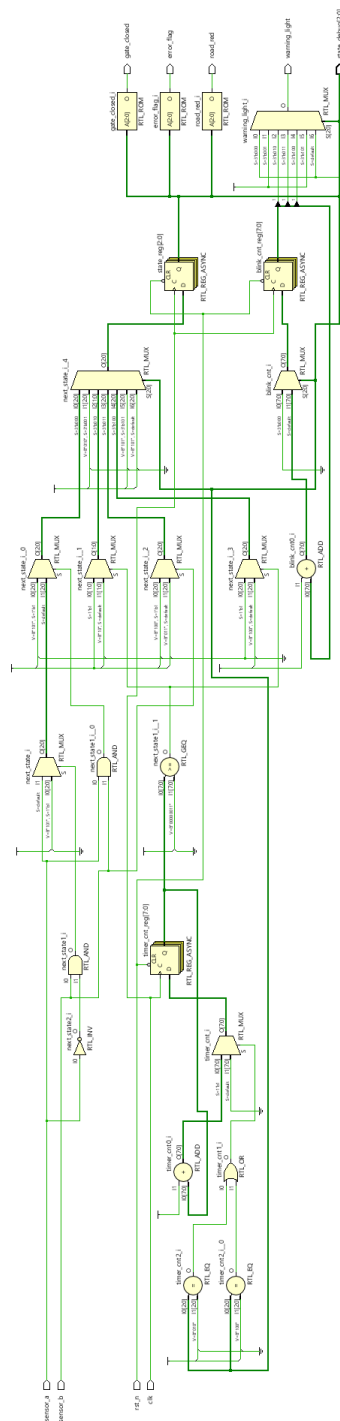


Figure 4: RTL-level synthesised schematic

Figure 5: RTL-level synthesised schematic



10. Conclusion

The Automatic Railway Level Crossing Controller was designed as a six-state Moore FSM, implemented in synthesisable Verilog, and verified against seven directed scenarios covering normal operation, back-to-back trains, extended crossing dwell time, and two classes of invalid/unexpected sensor input. All 29 self-checks passed and all six FSM states, including the fail-safe ERROR state, were exercised (100% state coverage). Generic logic synthesis with Yosys produced a compact 91-cell / 13-flip-flop netlist with zero structural check errors, confirming the design is synthesisable. The controller meets every functional requirement (a) through (j) of the project statement, and its fail-safe-by-design ERROR handling reflects the safety-critical nature of a real railway interlocking system.

Appendix A: Full Gate-Level (Technology-Mapped) Schematic

Figure 5 below is the complete post-synthesis gate-level netlist schematic (91 cells). It is included for completeness; due to its density it is best reviewed by opening `synth/schematic_gate_level.svg` at high zoom, or by inspecting the generated netlist file `railway_crossing_controller_netlist.v` directly.

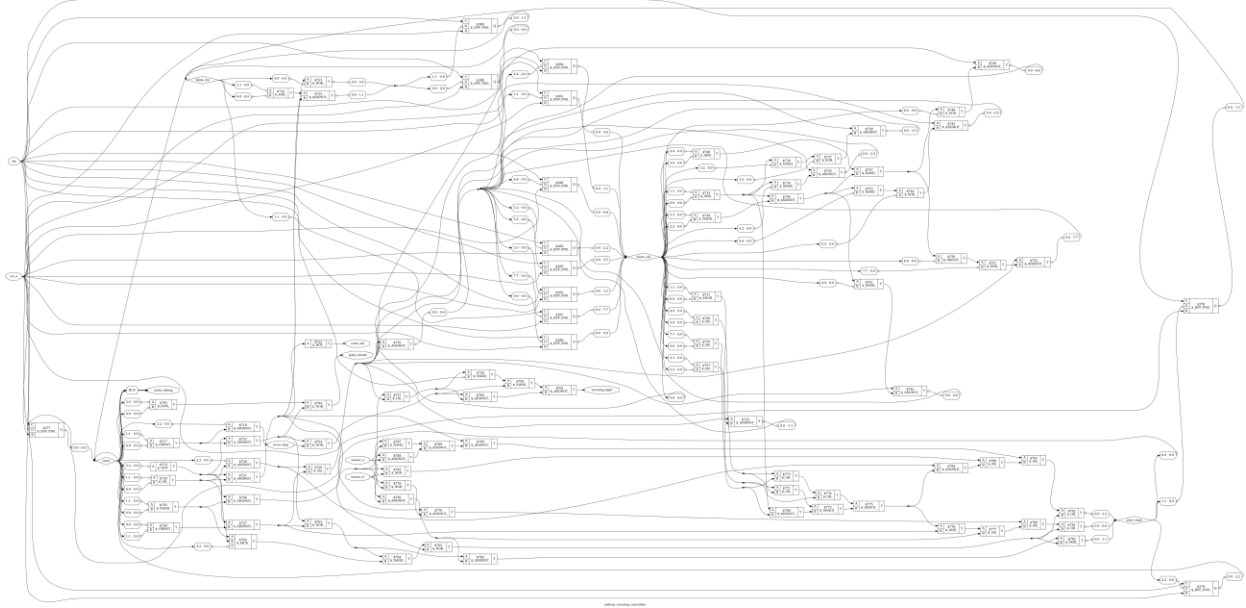


Figure 6: Full technology-mapped gate-level schematic

Appendix B: Toolchain Used

Tool	Version	Purpose
Icarus Verilog	12.0 (stable)	RTL simulation (compile + vvp execution)
Yosys	0.33	Generic logic synthesis, statistics report, schematic generation
GTKWave-equivalent (VCD + Python/matplotlib)	-	Waveform rendering from the simulator's VCD dump
Graphviz (dot)	-	FSM diagram and Yosys schematic rendering
Synopsys	-	Synthesis
Vivado	2018	RTL Schematic